

# Scripts for X-ray postprocessing with USPEX

Michele Galasso

January 6, 2026

## 1 Introduction

In this document I present a series of Python scripts used to postprocess the results of USPEX calculations when working with experimental X-ray diffraction data. The core of this collection of scripts is constituted by the scripts `powder_xrd_screening.py` and `scxrd_screening.py`, which allow the user to quickly sort a number of structures read from the USPEX output according to their degree of agreement with experimental X-ray data. The first script works with powder XRD data, while the second script works with single crystal XRD data. Both scripts can be used not only with USPEX output files, but also with any set of structures in the POSCAR format, for example, with structures taken from a database. The theory beyond the functioning of these two scripts is illustrated in appendices A and B.

The first 3 scripts take in input a number of files written by USPEX, while the remaining scripts work with structural information contained in CIF or POSCAR files. All script take in input some information about the experimental X-ray data as well. At the time of writing this document, the collection of scripts has been tested on the following Python versions: 3.6, 3.7, 3.8, 3.13.

## 2 `split_CIFs.py`

It splits the structures with lowest enthalpy of a variable composition USPEX run into multiple CIF files. The script takes the following arguments from command line:

1. the output file `extended_convex_hull` from USPEX
2. the output file `extended_convex_hull_POSCARS` from USPEX
3. the value of the external pressure used for the USPEX run

The script performs the following operations:

1. for each structure, it reads the parameters *enthalpy* and *fitness* from `extended_convex_hull` and the geometry from `extended_convex_hull_POSCARS`

2. it selects, for each reduced formula, the 5 structures with lowest enthalpy
3. it outputs the selected structures as CIF files in a new folder *results*

The name of each CIF file has the format `i_ID.fitness.enthalpy.iupacformula.pressure.symmetry.cif`, where:

- i** is a natural number which orders the output with increasing *fitness*
- ID** is the structure ID from the USPEX run
- fitness** is the *fitness* of the structure
- enthalpy** is the *enthalpy* of the structure
- iupacformula** is the *IUPAC formula* of the structure
- pressure** is the pressure used for the USPEX run
- symmetry** is the space group number, determined with tolerance 0.2 Å

**Example:** `python split_CIFs.py extended_convex_hull  
extended_convex_hull_POSCARS 50GPa`

### 3 sublattice\_split\_CIFs.py

It reads the results of a variable composition USPEX run, it removes all hydrogen atoms, and then it splits the structures with lowest enthalpy into multiple CIF files. The script takes the following arguments from command line:

1. the output file `extended_convex_hull` from USPEX
2. the output file `extended_convex_hull_POSCARS` from USPEX
3. the value of the external pressure used for the USPEX run

The script performs the following operations:

1. for each structure, it reads the parameters *enthalpy* and *fitness* from `extended_convex_hull` and the geometry from `extended_convex_hull_POSCARS`
2. it deletes all hydrogen atoms
3. it selects, for each reduced formula, the 5 structures with lowest enthalpy
4. it outputs the selected structures as CIF files in a new folder *results*

The name of each CIF file has the format `i_ID.fitness.enthalpy.iupacformula.pressure.symmetry.cif`, where:

- i** is a natural number which orders the output with increasing *fitness*
- ID** is the structure ID from the USPEX run

**fitness** is the *fitness* of the structure

**enthalpy** is the *enthalpy* of the structure

**iupacformula** is the *IUPAC formula* of the structure, with hydrogens

**pressure** is the pressure used for the USPEX run

**symmetry** is the space group number, determined with a tolerance of 0.2 Å and without hydrogens

**Example:** `python sublattice_split_CIFs.py extended_convex_hull  
extended_convex_hull_POSCARS 50GPa`

## 4 fixcomp\_split\_CIFs.py

It splits all the structures of a fixed composition USPEX run into multiple CIF files. The script takes the following arguments from command line:

1. the output file **Individuals** from USPEX
2. the output file **gatheredPOSCARS** from USPEX
3. the value of the external pressure used for the USPEX run

The script performs the following operations:

1. for each structure, it reads the parameter *enthalpy* from **Individuals** and the geometry from **gatheredPOSCARS**
2. it computes `real_fitness = enthalpy / total_number_of_atoms`
3. it outputs the structures as CIF files in a new folder *results*

The name of each CIF file has the format `i_ID_fitness_enthalpy_iupacformula_  
pressure_symmetry.cif`, where:

**i** is a natural number which orders the output with increasing `real_fitness`

**ID** is the structure ID from the USPEX run

**fitness** is the `real_fitness` of the structure

**enthalpy** is the *enthalpy* of the structure

**iupacformula** is the *IUPAC formula* of the structure

**pressure** is the pressure used for the USPEX run

**symmetry** is the space group number, determined with a tolerance of 0.2 Å

**Example:** `python fixcomp_split_CIFs.py Individuals gatheredPOSCARS  
50GPa`

## 5 powder\_xrd\_screening.py

The script screens a collection of structures to identify those that best match an experimental powder X-ray diffraction pattern. For the theory behind this script, see appendix A. In a few words, given an experimental pattern in input, the theoretical diffraction pattern is calculated for each candidate structure and a value  $F$  is computed. The smaller is  $F$ , the better is the agreement between theoretical and experimental patterns. The script contains the following input parameters:

1. the name of the file `gathered.POSCARS`, which is a collection of crystal structures in the POSCAR format
2. the name of the *spectrum file*, containing angles and intensities of the experimental powder diffraction pattern
3. the start and end angles for the computation of theoretical diffraction patterns
4. an array of coefficients  $k$  for performing isotropic deformations of the unit cell for all crystal structures
5. the value of  $\sigma$  for the Gaussian smearing of peaks, used for generating output pictures
6. the experimental wavelength
7. the parameter *match\_tol*, that is the tolerance for matching experimental peaks with theoretical peaks, in degrees
8. the parameter *th\_peaks\_penalty*, which controls the additional penalty given to non-matching theoretical peaks in our fitness function

The script performs the following operations:

1. for each structure, it reads the geometry from `gathered.POSCARS`
2. for each given value of  $k$ , it performs an isotropic deformation of the unit cell by multiplying each lattice parameter by  $k$
3. it computes the theoretical X-ray diffraction pattern of the so-obtained structure
4. it computes the disagreement  $F$  between theoretical and experimental diffraction patterns using our fitness function
5. for each structure and for the lowest value of  $F$  across all the coefficients  $k$ , it outputs a CIF file with the symmetrized structure (tolerance 0.2 Å)
6. for each structure and for the lowest value of  $F$  across all the coefficients  $k$ , it outputs a PNG graph with the theoretical and experimental diffraction patterns superimposed for comparison

7. for each structure, it outputs a PNG graph which shows the variation of  $F$  with  $k$

The name of each CIF and PNG file have the format `F_ID.iupacformula.k_symmetry`, where:

**F** is the disagreement  $F$  between theoretical and experimental diffraction patterns

**ID** is the comment provided in the first line of each POSCAR (if the POSCAR file is produced by USPEX, this is the structure ID in the USPEX run)

**iupacformula** is the *IUPAC formula* of the structure

**k** is the value of the coefficient  $k$  which gave the lowest disagreement  $F$

**symmetry** is the space group number, determined with a tolerance of 0.2 Å

**Example:** `python powder_xrd_screening.py`

## 6 scxrd\_screening.py

This script performs a screening of USPEX results, looking for the structures that best match an experimental single crystal X-ray spectrum. The fitness function for evaluating the agreement between theoretical and experimental spectra is the weighted R-factor (see appendix B). Notice that the USPEX run must have been performed at fixed cell parameters. The script contains the following input parameters:

1. the name of the `gatheredPOSCARS` file from the USPEX run
2. the name of the HKL file containing experimental reflections

The script outputs a file `results.txt` which contains a list of ordered structure IDs with their relative weighted R-factor, from the most fit to the least fit.

**Example:** `python scxrd_screening.py`

## 7 pareto\_picture.py

This script organizes and visualizes the results of a USPEX calculation in which multi-objective optimization of stability together with the agreement with an experimental powder XRD spectrum has been optimized. Lines 19-21 of the script contain the input parameters, which need to be tuned by the user:

1. the wavelength of the incident radiation in Å
2. the name of the working directory, which must contain the files `goodStructures` and `goodStructures_POSCARS` from the USPEX output, as well as the spectrum file

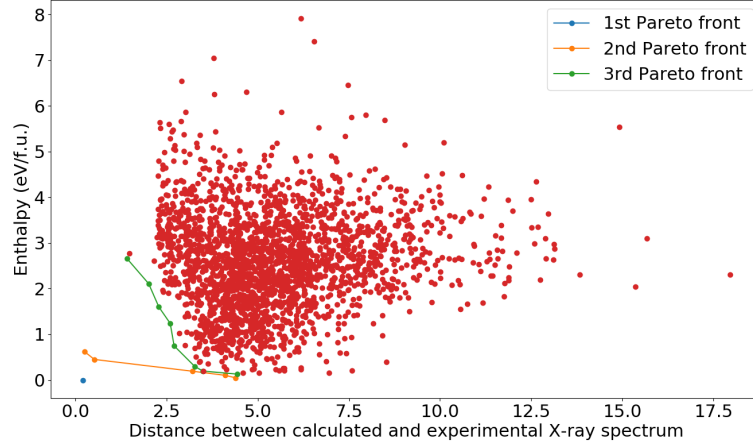


Figure 1: Graph produced by the script `pareto_picture.png` from the results of an USPEX run in which stability of the system  $\text{K}_4\text{Ta}_4\text{W}_4\text{O}_{24}$  has been optimized together with the agreement with an experimental powder XRD spectrum.

### 3. the name of the spectrum file

The output of the script is constituted by a picture, named `pareto.png`, displaying each structure on a plane where stability lies on the Y axis and the disagreement with the given spectrum lies on the X axis (see Fig. 1). On the picture, the first three Pareto fronts are shown. It is possible to focus on a specific area of the graph by specifying the intervals to display on the X and Y axes at lines 76-77 and by rerunning the script. In addition, the script creates a folder named `pareto_fronts` which contains, for each structure on the first three Pareto fronts, a structure file in the POSCAR format and a picture which visually compares the predicted powder XRD spectrum of the structure with the given spectrum contained in the spectrum file, as in Fig. 2. The name of these files have the format `rank_F_enthalpy_ID`, where:

**rank** is the progressive number of the Pareto front

**F** is the agreement F between simulated and given spectra

**enthalpy** is the enthalpy of the structure

**ID** is the structure ID from the USPEX run

**Example:** `python pareto_picture.py`

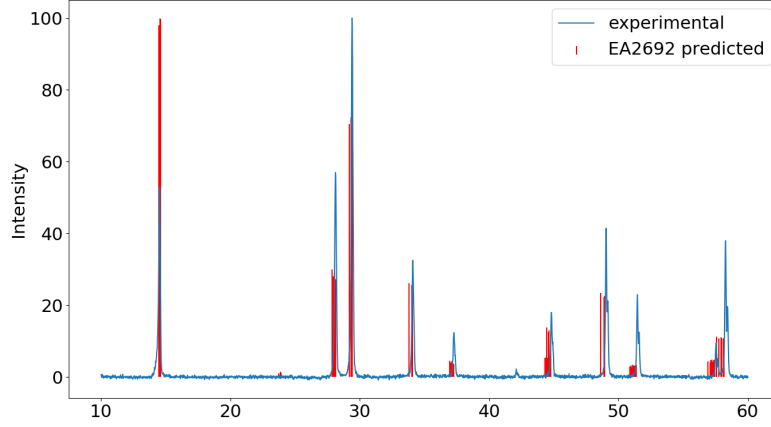


Figure 2: Graph produced by the script `pareto_picture.png` from the results of an USPEX run in which stability of the system  $\text{K}_4\text{Ta}_4\text{W}_4\text{O}_{24}$  has been optimized together with the agreement with an experimental powder XRD spectrum.

## 8 exclusion.py

It allows to quickly filter a multitude of CIF files, by removing those which have significant peaks in a user-defined exclusion region of the X-ray spectrum. The script takes the following arguments from command line:

1. the wavelength of the incident radiation in Å
2. the peak cut-off, in % of the maximum intensity
3. a number of intervals in degrees, defining the exclusion region
4. the extremes of the generated X-ray diffraction pattern in degrees

The script works in a folder with many CIF files, and performs the following:

1. it opens, one by one, all CIF files and it predicts the X-ray diffraction pattern of the structure according to the given wavelength
2. if the predicted pattern contains any peak in the exclusion regions that is bigger than the given cut-off, it deletes the CIF file

**Example:** `python exclusion.py --wavelength 0.6199 --cutoff 25 --lextremes 25,31 --rextremes 28,32 --minangle 5 --maxangle 60`

## 9 find\_peak.py

It allows to quickly filter a multitude of CIF files, by removing those which do not have significant peaks in all user-defined search intervals of the X-ray spectrum. The script takes the following arguments from command line:

1. the wavelength of the incident radiation in Å
2. the peak cut-off, in % of the maximum intensity
3. a number of search intervals in degrees, defining the search region
4. the extremes of the generated X-ray diffraction pattern in degrees

The script works in a folder with many CIF files, and performs the following:

1. it opens, one by one, all CIF files and it predicts the X-ray diffraction pattern of the structure according to the given wavelength
2. if there is at least one search region which does not contain any peak bigger than the given cut-off, it deletes the CIF file

**Example:** `python find_peak.py --wavelength 0.6199 --cutoff 15  
--lxtremes 25,31 --rextremes 28,32 --minangle 5 --maxangle 60`

## 10 change\_pressure.py

It translates the structures contained in many CIF files to a different pressure, by deforming the lattice parameters using a second order Taylor expansion of the Birch-Murnaghan equation of state. More details about the underlying theory can be found in appendix C. The script takes the following arguments from command line:

1. the initial pressure
2. the final pressure

The script works in a folder with many CIF files, and performs the following:

1. it reads the structures contained in all CIF files
2. it deforms the lattice parameters according to the new pressure
3. it creates new CIF files containing the deformed structures

The name of the new CIF files have the format **OLDNAME\_toNEWPRESSURE**, where:

**OLDNAME** is the name of the file with the structure at the initial pressure

**NEWPRESSURE** is the final pressure

**Example:** `python change_pressure.py 50 58`



## 11 relax\_new\_pressure.py

It allows an accurate translation of the structures contained in many CIF files to a different pressure, by relaxing them with VASP. The script works in a folder with many CIF files and it is meant to be run on a cluster. Before running it, add the following lines to your `.bashrc` file:

1. `export VASP_SCRIPT=PATH/TO/relax_new_pressure/run_vasp.py`
2. `export VASP_PP_PATH=PATH/TO/YOUR/pp/FOLDER`

Then tune the `relax_new_pressure.py` file with the desired new pressure and the appropriate VASP parameters for your calculation and run the python script on a computing node. By default, VASP relaxations are performed sequentially and every relaxation takes 4 computing cores. For more details about the implementation, have a look at the ASE VASP calculator.

## 12 single\_plot.py

This simple script is used to obtain a plot of the predicted powder XRD spectrum of a single structure against the experimental spectrum. It takes in input, at lines 12-14, the following three parameters:

1. the wavelength of the incident radiation in Å
2. the name of the file containing the geometry of the structure in POSCAR format
3. the name of the file containing the experimental spectrum

**Example:** `python single_plot.py`

# Appendices

## A Powder spectra comparison

We developed a code which computes, from the experimental powder diffraction pattern and a set of crystal structures, the degree of disagreement (fitness) of each structure with the experimental data.

In the case of powder X-ray diffraction patterns, for each peak we know the diffraction angle and the relative intensity, while the intensity of the highest peak in each spectrum has been conventionally given the value 100. We define a *match* between a theoretical and an experimental peak the case when the two peaks are less than *match\_tol* degrees apart, regardless of their intensities. The fitness between a calculated powder X-ray diffraction pattern and the experimental diffraction pattern is defined by the following fitness function

$$F = \sum_{i,j}^{match} \left( \frac{h_i^{exp} - h_j^{th}}{100} \right)^2 \left( \frac{h_i^{exp}}{100} \right)^2 + \sum_i^{exp\ rest} \left( \frac{h_i^{exp}}{100} \right)^2 + p^{th} \cdot \sum_i^{th\ rest} \left( \frac{h_i^{th}}{100} \right)^2$$

where the  $h_i$  are diffraction intensities. The first term of the fitness is a sum over the matched peaks, and each addend of this sum will be smaller the more the intensities of the two matched peaks are close to each other. The second term is a sum over the experimental peaks that are left after our matching, we call these peaks *experimental rest*. The third term, analogously, is a sum over the *theoretical rest*. The user-defined coefficient  $p^{th} > 1$  which appears in the last term introduces an additional penalty for non-matching theoretical peaks. It is clear that a low value of F gives a good agreement between calculated and experimental powder X-ray diffraction patterns, allowing a quick identification of promising candidates structures.

## B Single crystal spectra comparison

In the case of single crystal X-ray diffraction patterns, for each reflection we know the Miller indices  $(h, k, l)$ , its intensity  $I$  and its uncertainty of measure  $\sigma$ . As fitness we use the weighted R-factor, given by

$$wR = \sqrt{\frac{\sum \frac{1}{\sigma^2} (I_{exp} - I_{th})^2}{\sum \frac{1}{\sigma^2} I_{exp}^2}}$$

where the sums run over all the experimental reflections and theoretical reflections at the same Miller indices  $(h, k, l)$ . If an experimental reflection is not matched by any theoretical reflection an error is returned, while in the opposite case the unmatched theoretical reflection is simply ignored.

## C Pressure change

It often happens that the USPEX calculation and the experimental data are not exactly at the same pressure, but the two pressures are *close*, that is, no more than 20 GPa apart. In this case, our script `change_pressure.py` can be used to translate the theoretical structures to the experimental pressure.

We start from the Birch-Murnaghan equation of state

$$\Delta P = \frac{3B_0}{2} \left[ \left( \frac{V_0}{V} \right)^{\frac{7}{3}} - \left( \frac{V_0}{V} \right)^{\frac{5}{3}} \right] \left\{ 1 + \frac{3}{4} (B'_0 - 4) \left[ \left( \frac{V_0}{V} \right)^{\frac{2}{3}} - 1 \right] \right\}$$

where  $\Delta P$  is the pressure difference,  $V_0$  is the volume of the unit cell at the calculated pressure,  $V$  is the volume at the experimental pressure,  $B_0$  is the bulk modulus and  $B'_0$  is the derivative of the bulk modulus with respect to pressure. The parameters  $B_0$  and  $B'_0$  are user-defined and, if not explicitly specified, they default to the average values  $B_0 = 300$  and  $B'_0 = 3$ .

Assuming that, for small pressure variations,  $V_0/V$  is close to 1, we can approximate the Birch-Murnaghan equation of state to a second order Taylor expansion in  $V_0/V$  and get the volume  $V$  at the experimental pressure:

$$V = \frac{B_0(B'_0 - 1)}{B_0(B'_0 - 2) + \sqrt{B_0^2 + 2B_0(B'_0 - 1)\Delta P}} V_0$$

Then we define the following scaling factor

$$k = \sqrt[3]{\frac{B_0(B'_0 - 1)}{B_0(B'_0 - 2) + \sqrt{B_0^2 + 2B_0(B'_0 - 1)\Delta P}}}$$

that will be used to rescale the lattice parameters of all the calculated structures. After this rescaling, the relaxed structures are symmetrized with a tolerance of 0.2 Å and the corresponding CIF files are written.